

Gestione degli Accessi

- In JavaScript esiste un metodo chiamato isValidUser.
- Questo metodo può essere utilizzato per definire l'elenco delle **coppie username e password** che possono essere autorizzate, quando avviene l'autenticazione da parte del client
- Definiamoci il seguente oggetto Javascript, che utilizza il metodo isValidUser

```
const userManager = {  
  isValidUser: function(userName, password) {  
    if (userName === "user1" && password === "pas1") {  
      return true;  
    }  
    if (userName === "user2" && password === "pas2") {  
      return true;  
    }  
    return false;  
  }  
};
```

Istanza del Server

- Nella creazione dell'istanza del Server, è possibile negare l'accesso anonimo e definire gli accessi (ad esempio con username e password)

```
const server = new opcua.OPCUAServer({  
    port: 4334,  
    userManager,  
    allowAnonymous: false,  
    resourcePath: "/UA/MyLittleServer",  
});
```

Gestione dell'AddressSpace

- Una volta che il server è stato inizializzato, possiamo estendere l'AddressSpace predefinito con i nostri nodi personalizzati. Per fare questo dobbiamo seguire i seguenti passaggi:

1. Accedere all'AddressSpace :

```
const addressSpace = server.engine.addressSpace;
```

2. Accedere alla porzione riservata al Server (Namespace index=1) :

```
const namespace = addressSpace.getOwnNamespace();
```

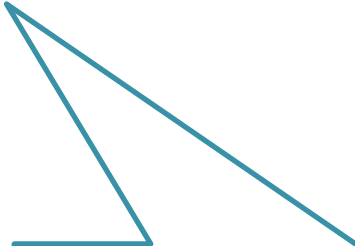
Gestione dell'AddressSpace

- Una volta avuto accesso al Namespace del Server è possibile creare tutti gli oggetti e/o tipi che si desidera, così come poi inizializzarli.
- Nel seguito supporremo di realizzare un semplice esempio:
 - Definiamo un sottotipo di BaseObjectType, di nome **"TemperatureSensorType"**, composto da DataVariable e da Proprietà (con Modelling Rules)
 - Definiamo un Folder sotto la cartella standard Objects, di nome **"MySensors"**
 - Istanziamo uno o più oggetti di tipo TemperatureSensorType dentro questa cartella MySensors
 - Assegniamo dei valori agli oggetti istanziati

Gestione dell'AddressSpace

- Creiamo nel Namespace di index 1 del Server, un nuovo ObjectType **TemperatureSensorType** definito come SubType di BaseObjectType

```
var temperatureSensorType = namespace.addObjectType({  
    browseName: "TemperatureSensorType"  
});
```



E' possibile inserire e inizializzare altri attributi, provate ad aggiungerli voi a piacere

Gestione dell'AddressSpace

- Arricchiamo l'ObjectType **TemperatureSensorType** con componenti e proprietà.
 - Ad esempio aggiungere una proprietà (Model) e una variabile (Temperature) all'ObjectType

```
var model = namespace.addVariable({  
    propertyOf: temperatureSensorType,  
    browseName: 'Model',  
    dataType: opcua.DataType.String,  
    modellingRule: 'Mandatory',  
})  
var temperature = namespace.addVariable({  
    componentOf: temperatureSensorType,  
    browseName: 'Temperature',  
    dataType: opcua.DataType.Double,  
    modellingRule: 'Mandatory',  
});
```

Nota: Ricordiamoci che possiamo specificare modellingRule come "Mandatory", "Optional", "PlaceholderMandatory", "PlaceholderOptional". Se manca modellingRule, si assume "Mandatory".

Gestione dell'AddressSpace

- Creiamo sotto lo standard Folder Objects, un Folder di nome "MySensors"

```
var objectFolder = namespace.addFolder(addressSpace.rootFolder.objects, {  
    browseName: "MySensors"  
});
```

- Istanziamo un nodo Object dall'ObjectType **TemperatureSensorType**, inserendo le istanze nella cartella **MySensors** creata ad hoc :

```
var temperatureSensor = temperatureSensorType.instantiate({  
    browseName: "MyTemperatureSensor",  
    organizedBy: objectFolder,  
});
```

Gestione dell'AddressSpace

- Per aggiornare il valore di un componente dell'Oggetto appena istanziato, è stato già visto come fare:

```
temperatureSensor.model.setValueFromSource({  
    dataType: opcua.DataType.String,  
    value: "1-Way",  
});  
  
setInterval(() => {  
    const value = 12 + 5 * Math.sin(Date.now() / 10000) + Math.random()*0.2;  
    temperatureSensor.temperature.setValueFromSource({ dataType: opcua.DataType.Double,  
        value });  
}, 100);
```

Gestione dell'AddressSpace

- E' possibile fare anche altre operazioni, ad esempio quella di creare un nuovo ObjectType come SubType di un tipo derivato, ad esempio del tipo precedentemente creato TemperatureSensorType

```
var specialTemperatureSensorType = namespace.addObjectType({  
    browseName: "specialTemperatureSensorType",  
    subTypeOf: temperatureSensorType  
});
```